PROJECT REPORT

RESOLVEIT

Multi-Tenant Customer Support & Grievance Redressal Platform

A Full-Stack Web Application developed using the MERN Stack

Submitted by

S. No.	Team member	Register number
1	TELLAPALLI AKHIL KUMAR	99230040782
2	KANIGIRI BHANU GANESH BABU	99230040975
3	SHAIK NAVEED AHMED	99230040949
4	GUNISETTY VENKATA GNANESWAR	99230040969
5	PATLOLLA VISHNU VARDHAN REDDY	99230040995
6	LAKKAKULA PENCHALA PRASAD	99230040982

Academic Project Report • 2026

ABSTRACT

ResolveIT is a multi-tenant customer support and grievance redressal web platform built using the MERN stack. It is designed to help multiple organizations manage customer complaints through separate, branded service desks while using a common application platform. The system provides a public customer desk, an internal company portal, and a platform-level administration area.

Customers can search frequently asked questions, submit complaints, track ticket progress using a reference identifier, communicate through ticket conversations, and provide a one-to-five-star satisfaction rating after resolution. Company developers manage company branding, categories, knowledge-base articles, employees, and ticket queues. Support employees work on assigned complaints, respond to customers, add private internal notes, and update ticket status. A Super Admin manages company onboarding and developer provisioning and can monitor platform-wide metrics.

The application uses React and Vite for the client interface, Node.js and Express for REST API services, and MongoDB with Mongoose for persistence. Authentication and access control are supported through JWT-based staff sessions, Google OAuth 2.0 customer sign-in, password hashing with bcryptjs, and role validation. The design emphasizes tenant-scoped data access, clearer ownership, workload distribution, service-level deadline visibility, and customer-safe progress tracking.

TABLE OF CONTENTS

S.NO	Content	Page.No
1	Introduction	3
2	Problem Statement	3
3	Objectives and Scope	3
4	Existing and Proposed Systems	4
5	System Requirements and Feasibility	4
6	System Architecture	5
7	Methodology and Workflow	6
8	Functional Modules	7
9	Database Design	8
10	Technology Stack	8
11	Implementation Details	9
12	Security and Privacy	10
13	Testing Strategy and Test Cases	11
14	Advantages and Applications	12
15	Limitations and Future Enhancements	12
16	Conclusion	13
17	References	13

1. INTRODUCTION

Customer support is an important function for organizations that provide products or services. Customers may report product defects, service interruptions, billing issues, delivery concerns, account problems, or requests for assistance. When these complaints arrive through disconnected channels such as email, phone calls, and informal messages, it can become difficult to identify ownership, maintain a consistent history, and communicate progress.

ResolveIT addresses this challenge through a centralized ticket-based support workflow. It provides a common platform on which multiple companies can operate independent support desks. Each organization can maintain its own identity, categories, FAQs, staff accounts, and complaint records. Customers interact with the public-facing service desk, while employees and company developers use a protected internal portal.

The system is described as multi-tenant because multiple organizations share the platform while their company-specific records and workflows remain logically separated. The platform also provides a Super Admin area for onboarding and monitoring the participating companies.

2. PROBLEM STATEMENT

- Fragmented complaint handling: customer issues may be submitted through disconnected channels, making ownership and follow-up difficult.
- Unclear responsibility: without structured routing, complaints may reach the wrong team or remain unassigned.
- Limited visibility: customers need a clear way to view case progress without being exposed to private staff discussions.
- Service-level pressure: organizations need to monitor deadlines, workload, and resolution performance consistently.

Therefore, a centralized and tenant-aware grievance management platform is required to organize complaint submission, classification, assignment, communication, resolution, and feedback.

3. OBJECTIVES AND SCOPE

3.1 Objectives

- Provide organizations with independent, branded customer resolution desks.
- Capture customer grievances and route them to the most relevant department or category.
- Support fair work distribution when multiple employees belong to the same department.
- Enable staff to track active cases, progress, and service-level agreement (SLA) deadlines from one workspace.
- Keep internal allocation notes private while exposing customer-safe progress updates.
- Measure service outcomes through ticket resolution tracking and customer satisfaction ratings.
- Provide platform administrators with company onboarding and cross-company operational visibility.

3.2 Scope

The scope includes customer-facing help desks, FAQ search, secure customer sign-in, complaint submission, ticket tracking, staff workspaces, internal notes, company-level configuration, employee management, platform

administration, and operational analytics. The repository documentation identifies four user roles: Super Admin, Company Developer, Support Employee, and Customer.

3.3 Intended Users

User	Primary purpose
Customer	Find help articles, submit complaints, track progress, communicate, and rate resolved tickets.
Support Employee	Work on assigned tickets, reply to customers, add private notes, and update status.
Company Developer	Configure a company portal, manage employees/categories/FAQs, and oversee company tickets.
Super Admin	Onboard companies, provision developers, and monitor platform-wide metrics.

4. EXISTING AND PROPOSED SYSTEMS

4.1 Existing System

In a conventional complaint-handling process, organizations may depend on separate inboxes, spreadsheets, phone calls, or unrelated tools. These approaches can make it difficult to maintain a single case history, identify the responsible employee, monitor response deadlines, and provide consistent customer updates.

4.2 Proposed System

ResolveIT proposes a centralized web application with company-specific public help centers and internal portals. Complaints are stored as tickets, categorized, assigned to staff, updated through a controlled lifecycle, and tracked against priority-based response deadlines. Customer-visible messages are separated from private internal notes. The platform-level administration area supports company onboarding and global monitoring.

4.3 Key Difference

Aspect	Conventional approach	ResolveIT approach
Complaint intake	Multiple disconnected channels	Company-specific public service desk
Ownership	Manual or unclear assignment	Department/category-aware ticket workflow
Progress visibility	Follow-up often requires direct contact	Reference-based ticket tracking
Team collaboration	Separate conversations and records	Ticket thread with private internal notes
Performance monitoring	Manual review	Dashboards and SLA-related metrics
Organization support	Often configured for one organization	Multiple tenant-aware company portals

5. SYSTEM REQUIREMENTS AND FEASIBILITY

5.1 Functional Requirements

- Users must be able to authenticate according to their role and access permissions.
- Customers must be able to browse company FAQs and submit complaints.
- The system must generate a ticket reference identifier and allow ticket tracking.

- Authorized staff must be able to view ticket queues and update ticket status or priority.
- Staff must be able to post customer-visible replies and private internal notes.
- Company developers must be able to manage employees, categories, branding, and knowledge-base articles.
- Super Admins must be able to create and manage companies and developer accounts.
- The platform should expose company-level and platform-level ticket metrics.

5.2 Non-Functional Requirements

- Security: authentication, role checks, password hashing, and tenant-scoped access.
- Usability: clear navigation for customers, employees, company developers, and administrators.
- Maintainability: separation of frontend pages, backend routes, models, and middleware.
- Scalability: tenant-aware architecture intended to support multiple organizations.
- Availability: web deployment architecture using Vercel is identified in the project presentation.
- Responsiveness: the frontend uses a responsive CSS design system.

5.3 Feasibility

Technical feasibility is supported by the use of widely adopted web technologies: React, Node.js, Express, and MongoDB. Operational feasibility comes from role-specific interfaces that reflect the tasks of customers and support teams. Economic feasibility is supported by the use of open-source development tools and cloud-hosted services; actual operating costs depend on deployment plans, database usage, and notification volume.

6. SYSTEM ARCHITECTURE

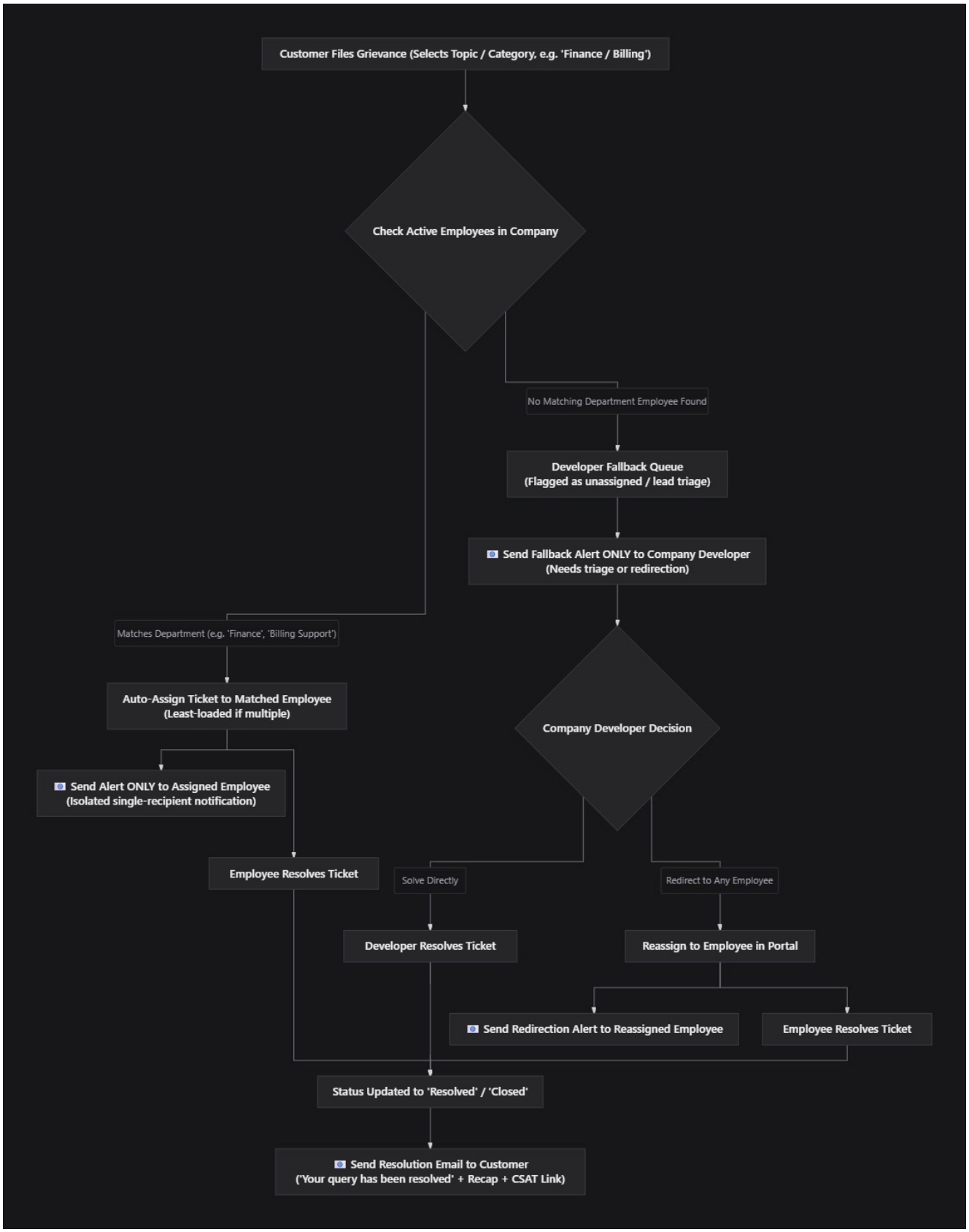
ResolveIT follows a layered client-server architecture. The React single-page application communicates with the Express REST API. Authentication and role-validation middleware protect routes, business services implement ticket workflows, and Mongoose models persist records in MongoDB.

Layer	Technology / responsibility
Client layer	React 19, Vite, React Router v7, Lucide React, responsive CSS
API / application layer	Node.js and Express REST API; tenant-scoped routes
Security and identity	JWT sessions, role validation, Google OAuth 2.0, bcryptjs
Business services	Ticket routing, assignment/load balancing, SLA tracking, email notifications
Data layer	MongoDB Atlas and Mongoose models
Deployment	Vercel frontend deployment with API forwarding/serverless handler as described in the presentation

6.1 Architecture Flow

1. The user opens the relevant company service desk or internal portal.
2. The React frontend renders the interface based on the route and authenticated role.
3. Requests are sent to the Express API endpoints.
4. Authentication and authorization middleware validate the session and permitted operation.
5. Business logic processes the request, including ticket creation, routing, status changes, or analytics.
6. Mongoose interacts with MongoDB to read or update the corresponding records.
7. The API returns a response and the frontend updates the relevant screen.

6.2 High-Level Architecture Diagram



7. METHODOLOGY AND WORKFLOW

7.1 End-to-End Ticket Lifecycle

Stage	Description
Submit	The customer selects a category and files a complaint through the company-specific help desk.
Classify	The topic or category is evaluated to support department routing.
Assign	The ticket is assigned to a matching employee using the least-active-caseload approach described in the presentation.
Resolve	Staff communicate with the customer, update the ticket status, and work against SLA deadlines.
Close and review	After resolution, the customer can provide a one-to-five-star customer satisfaction rating.

Fallback handling: when no employee matches a department, the presentation specifies that the case enters the Company Developer triage queue.

7.2 Ticket Statuses

The repository README identifies the ticket statuses Open, In Progress, Resolved, and Closed. These statuses represent the progression of a complaint from initial intake through active handling and final closure.

7.3 Customer and Staff Communication

The ticket conversation thread supports public replies that are visible to the customer and private internal notes intended for the support team. This separation helps staff coordinate without exposing internal allocation or discussion details on the public tracking view.

8. FUNCTIONAL MODULES

8.1 Super Admin Module

- View global analytics and platform-wide ticket metrics.
- Create and manage company records and company slugs.
- Classify company industries.
- Create or assign Company Developer accounts.
- Monitor ticket volume, response times, and resolution rates across companies.

8.2 Company Developer Module

- Configure company branding and theme.
- Create and manage department employees/support agents.
- Manage FAQs and knowledge-base articles.
- Review and resolve triage cases.
- Reassign or redirect tickets when required.
- Monitor company-level ticket queues and SLA compliance.

8.3 Support Employee Module

- Access assigned case workspace and department ticket queues.
- View ticket details and conversation history.
- Send customer-visible replies.
- Add private notes for internal coordination.
- Update ticket status and work toward priority-based deadlines.

8.4 Customer Module

- Open the public service desk for a specific company.
- Search FAQs and knowledge-base content.
- Sign in using Google authentication.
- Submit complaints and receive a ticket reference code.
- Track complaint progress and view public milestones.
- Communicate through the ticket thread and rate resolved cases from one to five stars.

8.5 Knowledge Base and FAQ Module

The knowledge-base module stores company-specific articles and FAQs. Customers can search for information before creating a complaint, while authorized staff can create, edit, and delete articles. This supports self-service and can reduce repetitive support requests.

8.6 Analytics and SLA Module

The platform includes ticket-volume and service-performance views. The repository README identifies company analytics such as ticket volume, average resolution time, and rating metrics. The presentation additionally describes SLA deadline tracking and workload visibility.

9. DATABASE DESIGN

The application uses MongoDB with Mongoose object data modelling. The repository identifies five core Mongoose models: User, Company, Ticket, Category, and Article.

Collection / Model	Purpose
User	Stores staff/customer identity, credentials, roles, and account-related data.
Company	Stores tenant profile, company slug, industry, and branding configuration.
Ticket	Stores complaint details, status, priority, assignment, conversation-related data, and SLA information.
Category	Stores complaint categories used to support classification and department routing.
Article	Stores company knowledge-base content and FAQ articles.

9.1 Logical Relationships

- A Company is associated with its users, categories, articles, and tickets.

- A User may represent a customer, support employee, Company Developer, or Super Admin depending on role.
- A Ticket belongs to a company and is associated with the customer who submitted it.
- A Ticket may be assigned to a support employee and may contain a sequence of public messages and private notes.
- A Category helps classify tickets and guide routing.
- An Article is associated with a company and is presented through that company's public help desk.

The above describes the logical data relationships from the repository and presentation. Exact field names, indexes, validation rules, and schema cardinalities should be confirmed against the implementation source files before a database-level design submission.

10. TECHNOLOGY STACK

Technology	Use in the project
React 19	Builds reusable frontend components and role-specific pages.
Vite	Frontend development server and build tool.
React Router v7	Handles client-side navigation and protected route flows.
Lucide React	Provides interface icons.
Vanilla CSS	Implements styling and responsive design.
Node.js	JavaScript runtime for backend services.
Express.js	Defines REST API routes and middleware.
MongoDB Atlas	Cloud-hosted document database.
Mongoose	Defines models and manages database operations.
JWT	Supports staff session authentication.
Google OAuth 2.0	Supports customer Google sign-in.
bcryptjs	Hashes passwords before storage.
EmailJS REST API	Identified in the project presentation for role-specific transactional notifications.
Vercel	Deployment platform identified in the presentation.

11. IMPLEMENTATION DETAILS

11.1 Frontend Structure

The frontend is organized into shared components, authentication context, and page groups. The repository README identifies separate page areas for platform administration, company staff portals, and public customer service pages. The App component configures client-side routing, while the global stylesheet provides the design system.

Frontend area	Responsibilities
components/	Shared interface components and protected-route wrapper.
context/	Authentication context, user state, tokens, and session handling.
pages/admin/	Super Admin dashboard, company management, and developer management.
pages/portal/	Company dashboard, tickets, agents, and customization.
pages/service/	Public help desk, complaint form, tracker, and customer history.

LandingPage.jsx	Main platform landing page.

11.2 Backend Structure

Backend area	Responsibilities
config/	Database connection setup.
middleware/	JWT authentication and role-check middleware.
models/	Mongoose schemas for User, Company, Ticket, Category, and Article.
routes/	Express route handlers for authentication, administration, companies, tickets, and related features.
app.js	Express application setup and middleware configuration.
index.js	Backend server entry point.
seedData.js	Initial sample data setup for demonstration.
resetDb.js	Database reset and re-seeding utility.

11.3 API Overview

API group	Representative endpoints	Purpose
Authentication	POST /api/auth/login; POST /api/auth/google; GET /api/auth/me	Staff login, customer Google authentication, and current-session retrieval.
Super Admin	GET /api/admin/stats; GET/POST /api/admin/companies; GET/POST /api/admin/developers	Platform metrics, company management, and developer provisioning.
Company portal	GET /api/companies/:slug; PUT /api/companies/:id; employee endpoints	Company information, branding, and employee management.
Tickets	GET/POST /api/tickets; PUT /api/tickets/:id; ticket messages and ratings	Ticket creation, retrieval, updating, messaging, and satisfaction rating.
Knowledge base	GET/POST /api/articles; PUT/DELETE /api/articles/:id	FAQ and article retrieval and management.
Analytics	GET /api/analytics/company/:companyId	Company ticket and service-performance metrics.

11.4 Local Setup (as documented in the repository)

The repository README describes a Node.js and npm development environment and a MongoDB Atlas or local MongoDB database. A typical setup sequence is:

8. Clone the project repository and open the project directory.
9. Install dependencies using npm install.
10. Create a local .env file based on .env.example and configure the database URI, JWT secret, Google OAuth client ID, port, and client URL.
11. Run the sample-data seed script if demonstration data is required.
12. Start the backend and frontend using the npm scripts documented in package.json.

Configuration values such as database connection strings, OAuth credentials, and JWT secrets must be kept private and must not be committed to a public repository.

12. SECURITY AND PRIVACY

- Authentication: JWT-based sessions are used for staff access, while Google OAuth 2.0 is identified for customer sign-in.
- Password protection: bcryptjs is used for password hashing.
- Role-based authorization: middleware validates the user's role before allowing protected operations.
- Tenant isolation: company-specific routes and data access are designed to keep one tenant's records separate from another tenant's records.
- Private notes: internal staff notes are distinguished from customer-visible replies.
- Public tracking: the customer-facing tracker exposes progress information without revealing internal staff allocation or private notes.

Security depends on correct implementation of server-side authorization and tenant filters for every relevant operation. The project should be tested for cross-tenant access, unauthorized role access, and accidental exposure of private notes.

13. TESTING STRATEGY AND TEST CASES

Testing should verify that each role can perform its permitted actions and cannot access functions outside its authorization. The cases below are a suggested validation checklist derived from the documented features; they are not presented as measured test execution results.

Test ID	Scenario	Expected result
TC-01	Staff user submits valid login credentials.	Authenticated session is established and the correct portal is shown.
TC-02	Customer completes Google sign-in.	Customer authentication is accepted after OAuth verification.
TC-03	Customer opens a company help desk.	Only the selected company's public information is displayed.
TC-04	Customer submits a valid complaint.	A ticket is created and a reference identifier is returned.
TC-05	Customer tracks a valid ticket reference.	Customer-safe ticket progress is displayed.
TC-06	Employee opens assigned ticket queue.	Authorized tickets are listed for the employee/company context.
TC-07	Employee posts a public reply.	The reply appears in the customer conversation thread.
TC-08	Employee adds a private note.	The note is visible to authorized staff and hidden from customers.
TC-09	Employee updates ticket status.	The ticket status is updated and reflected in the relevant views.
TC-10	Customer rates a resolved ticket.	A one-to-five-star rating is recorded.
TC-11	Company Developer edits a knowledge-base article.	The updated article is available on the company help desk.
TC-12	User attempts an operation outside their role.	The API rejects unauthorized access.
TC-13	User from one tenant attempts to access another tenant's ticket.	The request is denied or returns no unauthorized data.
TC-14	Super Admin creates a company developer account.	The developer account is created/assigned to the intended company.

13.1 Testing Types

- Unit testing: validate individual functions such as ticket validation, assignment selection, and role checks.
- Integration testing: verify communication among frontend, API routes, authentication middleware, and MongoDB.
- System testing: validate end-to-end workflows for each user role.
- Security testing: test unauthorized access, tenant isolation, session handling, and private-note visibility.
- Usability testing: confirm that customers and staff can complete common tasks with clear navigation.

14. ADVANTAGES AND APPLICATIONS

14.1 Advantages

- Clear ownership: department-aware routing helps direct complaints to the appropriate team.
- Balanced workload: least-active-caseload assignment is intended to distribute open tickets among eligible employees.
- Customer privacy: public tracking separates customer-visible progress from internal allocation and notes.
- Faster communication: targeted notifications can alert responsible employees and relevant roles.
- Operational visibility: dashboards provide ticket queues, workload, and resolution metrics.
- Consistent service: SLA deadlines and customer satisfaction feedback support service-quality monitoring.
- Multi-company support: multiple organizations can operate branded service desks on a shared platform.

14.2 Applications

- E-commerce businesses handling orders, returns, delivery, and payment complaints.
- Software and IT service providers managing technical support requests.
- Educational institutions handling student and parent grievances.
- Healthcare service organizations managing non-emergency service enquiries and feedback.
- Service companies that need department-based complaint routing and resolution tracking.

15. LIMITATIONS AND FUTURE ENHANCEMENTS

15.1 Considerations / Limitations

- The quality of automated or category-based routing depends on accurate category configuration and employee-department mapping.
- Service-level reporting is useful only when ticket timestamps, priorities, and status transitions are consistently maintained.
- Multi-tenant security requires thorough server-side authorization and tenant filtering across all API operations.
- Email delivery and third-party authentication depend on valid external service configuration and availability.
- Actual performance, availability, and scalability depend on hosting plans, database capacity, and operational monitoring.

15.2 Future Enhancements

- Introduce AI-assisted complaint classification and suggested department routing.
- Add sentiment analysis to help prioritize urgent or dissatisfied customer interactions.
- Provide multilingual interfaces and automatic translation for customer-staff communication.
- Add advanced SLA escalation rules and automated reminders for approaching or overdue tickets.
- Develop richer analytics with trend charts, department comparisons, and exportable reports.
- Provide attachment support with file validation, malware scanning, and secure storage.
- Add a mobile application or progressive web app for customers and support employees.
- Integrate additional channels such as email intake, messaging platforms, and web chat.

16. CONCLUSION

ResolveIT presents a centralized approach to customer complaint handling through a multi-tenant, role-based support platform. It brings together customer-facing help desks, company staff portals, ticket workflows, knowledge-base content, private internal collaboration, SLA visibility, and platform-level administration.

By organizing complaints as trackable tickets and associating them with companies, categories, and responsible employees, the system aims to improve accountability and make progress easier to monitor. Customer-facing tracking and satisfaction ratings provide a feedback loop, while company and platform dashboards support operational oversight.

The project demonstrates the use of the MERN stack to build a modular full-stack application. Further development can strengthen routing intelligence, analytics, escalation automation, multilingual support, and integrations with additional customer communication channels.

REFERENCES

13. ResolveIT project repository supplied for this report: https://github.com/tellapallyakhil/ICL-Project.git
14. ResolveIT project presentation supplied by the project team: “ICL Project – Full-Stack Web Application”, 2026.
15. React documentation: https://react.dev/
16. Vite documentation: https://vite.dev/guide/
17. Node.js documentation: https://nodejs.org/docs/
18. Express documentation: https://expressjs.com/
19. MongoDB documentation: https://www.mongodb.com/docs/
20. Mongoose documentation: https://mongoosejs.com/docs/
21. JSON Web Token introduction: https://jwt.io/introduction
22. Google Identity / OAuth documentation: https://developers.google.com/identity